Assignment 1

Batch Code: CSA0307D

S.No	Reg No	Name
23	192524514D	V. Mukesh

A. What is tail recursion? (5 marks)

Definition: Tail recursion is a form of recursion in which the recursive call is the **last operation** a function performs. Nothing is left to do after the call returns, such as multiplying, adding or printing.

Why it matters: Since no work is pending after the call, the compiler can reuse the current stack frame instead of creating a new one (**tail call optimization**). This gives O(1) stack space instead of O(n), and avoids stack overflow for deep recursion.

Example: Factorial

Non-tail recursive (multiplication happens after the call returns):

```
int fact(int n) {
    if (n == 0) return 1;
    return n * fact(n - 1);
}
```

Tail recursive (uses an accumulator):

```
int factTail(int n, int acc) {
    if (n == 0) return acc;
    return factTail(n - 1, n * acc);
}
```

Call: factTail(5, 1) returns 120.

Trace of factTail(3, 1): factTail(3,1) → factTail(2,3) → factTail(1,6) → factTail(0,6) → returns 6.

Feature	Normal recursion	Tail recursion
Work after call	Yes	No
Stack space	O(n)	O(1) with optimization
Conversion to loop	Harder	Easy

B. Design an algorithm for merging two arrays efficiently (10 marks)

Problem: Given two **sorted** arrays A[m] and B[n], produce one sorted array C[m+n].

Idea: Use the two-pointer technique, as in the merge step of merge sort. Compare the current elements of both arrays, copy the smaller one into C, and advance that array's pointer. Finally, copy whatever remains. Each element is visited only once.

Algorithm

```
MERGE(A, m, B, n, C)
1. i <- 0, j <- 0, k <- 0
2. while i < m AND j < n
       if A[i] <= B[j]
           C[k] <- A[i]; i <- i + 1
       else
           C[k] <- B[j]; j <- j + 1
       k <- k + 1
3. while i < m:  C[k] <- A[i]; i <- i + 1; k <- k + 1
4. while j < n:  C[k] <- B[j]; j <- j + 1; k <- k + 1
5. Return C
```

C Program

```
#include <stdio.h>

void merge(int A[], int m, int B[], int n, int C[]) {
    int i = 0, j = 0, k = 0;

    while (i < m && j < n) {
        if (A[i] <= B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];
    }
    while (i < m) C[k++] = A[i++];
    while (j < n) C[k++] = B[j++];
}

int main() {
    int A[] = {1, 3, 5, 7};
    int B[] = {2, 4, 6, 8, 10};
    int m = 4, n = 5, C[9];

    merge(A, m, B, n, C);

    printf("Merged array: ");
    for (int i = 0; i < m + n; i++)
        printf("%d ", C[i]);
    return 0;
}
```

Output: `Merged array: 1 2 3 4 5 6 7 8 10`

Dry run (A = {1,3,5}, B = {2,4})

Step	i	j	Compare	C
1	0	0	1 ≤ 2	1
2	1	0	3 > 2	1 2
3	1	1	3 ≤ 4	1 2 3
4	2	1	5 > 4	1 2 3 4
5	B finished	copy rest of A		1 2 3 4 5

Complexity analysis

- **Time:** O(m + n). Each element is copied exactly once.
- **Space:** O(m + n) for the output array C. An in-place merge is possible (e.g., the gap method) but is slower or more complex.
- **Best case:** O(min(m, n)) comparisons, when one array is entirely smaller than the other. **Worst case:** m + n – 1 comparisons.

Note: If the arrays are unsorted, either sort each first at O(m log m + n log n) and then merge, or simply concatenate if sorted order isn't needed (O(m + n)).

C. Importance of data structures in cybersecurity (5 marks)

Data structures let security systems store, search and analyze huge volumes of data quickly, which is essential for detecting threats in real time.

- **Hash tables:** used for password storage (salted hashes), file integrity checks and malware signature lookup in O(1) average time. Hash functions such as SHA-256 verify data integrity.
- **Trees (Tries, Merkle trees, B-trees):** tries support fast IP and URL prefix matching in firewalls and routers. Merkle trees verify data integrity in blockchains and secure updates. B-trees index large log and audit databases.
- **Graphs:** model networks to detect attack paths, lateral movement, botnet relationships and vulnerable nodes.
- **Stacks and queues:** stacks track function calls (important for analyzing buffer overflow and stack-smashing attacks). Queues manage packet processing in Intrusion Detection Systems (IDS) and handle log events in order.
- **Bloom filters:** space-efficient checks for known malicious URLs or IPs, with no false negatives.
- **Arrays and linked lists:** used for buffers, packet storage, access control lists (ACLs) and cryptographic operations on blocks of data.
- **Heaps and priority queues:** prioritize alerts and threats by severity.

Conclusion: Choosing the right data structure improves speed, memory efficiency and scalability of security tools such as firewalls, antivirus engines, IDS/IPS and encryption systems. Poor choices can create performance bottlenecks or exploitable weaknesses.